Operating Systems: Deadlocks & Synchronization — Concurrency and the Critical Section Problem

1.1 The Critical Section Problem

A critical section is a segment of code in which a process accesses shared resources, such as shared variables, files, or tables. When one process is executing in its critical section, no other process is allowed to execute in its critical section. Concurrent access to shared data may result in data inconsistency, termed a race condition.

A valid solution to the critical section problem must satisfy three mandatory criteria: (1) Mutual Exclusion: if process Pi is executing in its critical section, no other processes can execute in their critical sections; (2) Progress: if no process is in its critical section and some processes wish to enter, only those processes not in their remainder sections can participate in deciding who enters next; (3) Bounded Waiting: there must exist a bound on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter.

1.2 Peterson's Software Solution

Peterson's solution is a classic software-based solution to the critical section problem restricted to two processes, P0 and P1. It requires two shared data items: an integer variable 'turn' and a boolean array 'flag[2]'.

The variable 'turn' indicates whose turn it is to enter the critical section. The 'flag' array indicates if a process is ready to enter its critical section. Process Pi sets flag[i] = true and turn = j. Pi waits while flag[j] == true and turn == j. Peterson's algorithm provably guarantees mutual exclusion, progress, and bounded waiting on sequential-consistency hardware architectures.

Operating Systems: Deadlocks & Synchronization — Hardware Synchronization & Semaphores

2.1 Hardware Support for Synchronization

Modern computer architectures provide special hardware instructions that allow software to test and modify the content of a word, or to swap the contents of two words, atomically (as one uninterruptible unit). Two fundamental primitives are TestAndSet and CompareAndSwap.

A Mutex (mutual exclusion) lock is the simplest synchronization tool. A process must acquire the lock before entering its critical section and release it when exiting. Mutex locks that require busy waiting are called spinlocks; they avoid context-switch overhead when lock holding times are brief.

2.2 Semaphores: Counting & Binary

A Semaphore is a synchronization tool that provides more sophisticated ways for process synchronization. A semaphore S is an integer variable accessed only through two standard atomic operations: wait() (historically termed P) and signal() (historically termed V).

In counting semaphores, the value can range over an unrestricted integer domain, controlling access to a resource with a finite number of instances. In binary semaphores, the value can range only between 0 and 1, functioning identically to a mutex lock. To avoid busy waiting in semaphores, the wait() operation suspends the calling process, placing it on a waiting list associated with the semaphore.

Operating Systems: Deadlocks & Synchronization — Classic Problems of Synchronization

3.1 The Bounded-Buffer (Producer-Consumer) Problem

The Bounded-Buffer problem represents a common paradigm for cooperating processes. A producer process produces information that is consumed by a consumer process. The pool consists of n buffers, each capable of holding one item.

Three semaphores are used: a mutex semaphore initialized to 1 for mutual exclusion; an 'empty' counting semaphore initialized to n representing the count of empty buffer slots; and a 'full' counting semaphore initialized to 0 representing the count of filled buffer slots. The producer waits on empty and mutex, deposits an item, and signals mutex and full.

3.2 Readers-Writers and Dining Philosophers

In the Readers-Writers problem, a database is shared among concurrent processes. Readers only read the database, whereas writers read and write. If a writer and another process access the database simultaneously, chaos ensues. Solutions require that multiple readers can access concurrently, but writers demand exclusive mutual exclusion.

The Dining Philosophers problem illustrates the challenges of allocating several resources among several processes in a deadlock-free and starvation-free manner. Five philosophers sit around a table with five chopsticks. Each philosopher needs both the left and right chopsticks to eat. If every philosopher picks up their left chopstick simultaneously, a circular wait deadlock occurs.

Operating Systems: Deadlocks & Synchronization — Deadlock System Model and Characterization

4.1 The Deadlock Problem

In a multiprogramming environment, several processes may compete for a finite number of resources. A process requests resources; if the resources are not available at that moment, the process enters a waiting state. A deadlock occurs when every process in a set of processes is waiting for an event that can be caused only by another process in the set.

Resources in a computer system are categorized into resource types (e.g., CPU cycles, memory space, I/O devices), each consisting of a number of identical instances.

4.2 The Four Coffman Conditions & Resource Graphs

A deadlock situation can arise if and only if the following four Coffman conditions hold simultaneously in a system: (1) Mutual Exclusion: at least one resource must be held in a nonshareable mode; (2) Hold and Wait: a process must be holding at least one resource and waiting to acquire additional resources held by other processes; (3) No Preemption: resources cannot be preempted; (4) Circular Wait: a closed chain of processes exists such that each process holds at least one resource that is needed by the next process in the chain.

Deadlocks can be described precisely in terms of a directed graph called a Resource-Allocation Graph (RAG). The graph consists of vertices representing processes (circles) and resource types (rectangles), and directed edges representing requests (P -> R) and assignments (R -> P). If the graph contains no cycles, the system is definitely not deadlocked. If the graph contains a cycle and each resource type has only one instance, a deadlock definitely exists.

Operating Systems: Deadlocks & Synchronization — Deadlock Prevention, Avoidance, and Recovery

5.1 Deadlock Prevention

Deadlock prevention provides a set of methods for ensuring that at least one of the four necessary Coffman conditions cannot hold. By preventing mutual exclusion (e.g., using spooling for printers), hold and wait (requiring processes to request all resources at once), no preemption (preempting held resources if an additional request cannot be granted), or circular wait (imposing a global linear ordering on all resource types), deadlock is impossible.

However, deadlock prevention generally results in low device utilization and reduced system throughput due to stringent restrictions on resource requests.

5.2 Deadlock Avoidance and Banker's Algorithm

Deadlock avoidance requires that the operating system be given additional advance information concerning which resources a process will request and use during its lifetime. A state is safe if the system can allocate resources to each process in some order without deadlocking.

Dijkstra's Banker's Algorithm is a classic deadlock-avoidance algorithm for systems with multiple instances of each resource type. It maintains data structures: Available (vector of length m), Max (n x m matrix), Allocation (n x m matrix), and Need (n x m matrix where Need[i][j] = Max[i][j] - Allocation[i][j]). When a process requests resources, the algorithm simulates whether granting the request leaves the system in a safe state; if not, the process must wait. Recovery from deadlock involves process termination or resource preemption with rollback.